

Synchronous FIFO UVM Verification

Using Verilator & DPI-C for Functional Coverage

GitHub: [NguyenThanhTai0307 / Synchronous-FIFO-UVM](#)

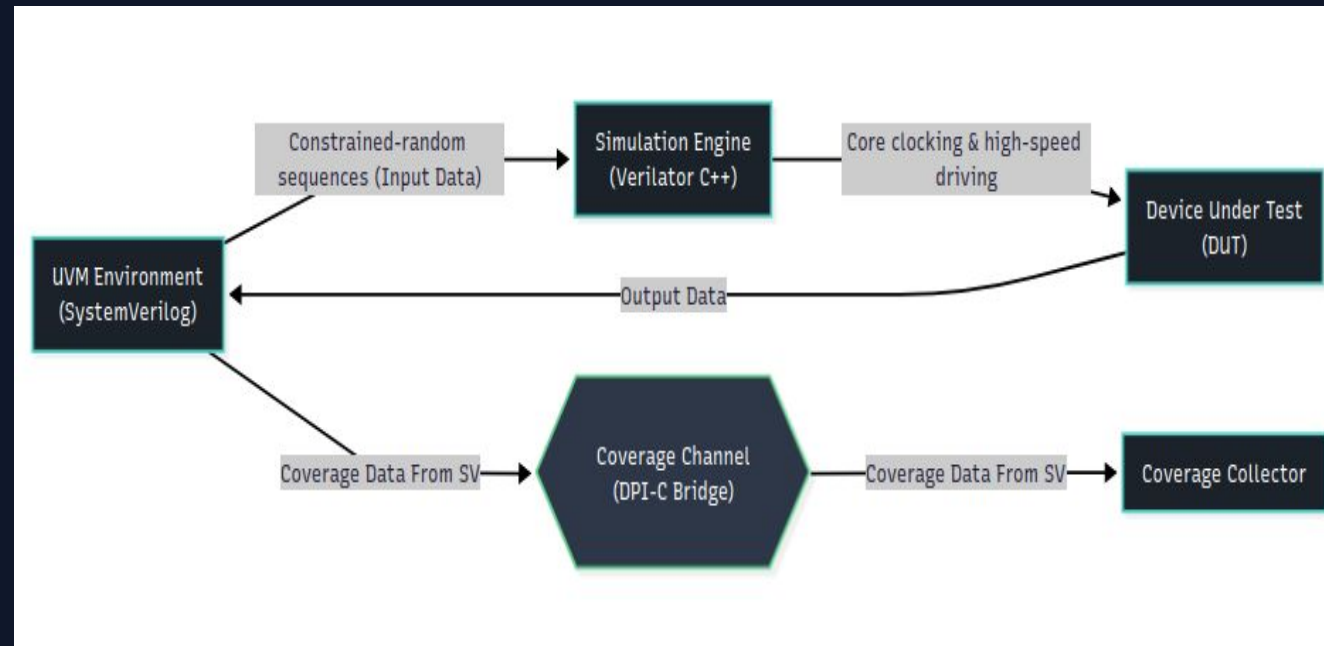
Project Objectives

Bridging industry-standard Universal Verification Methodology with high-performance open-source simulation tools, optimized for execution speed.

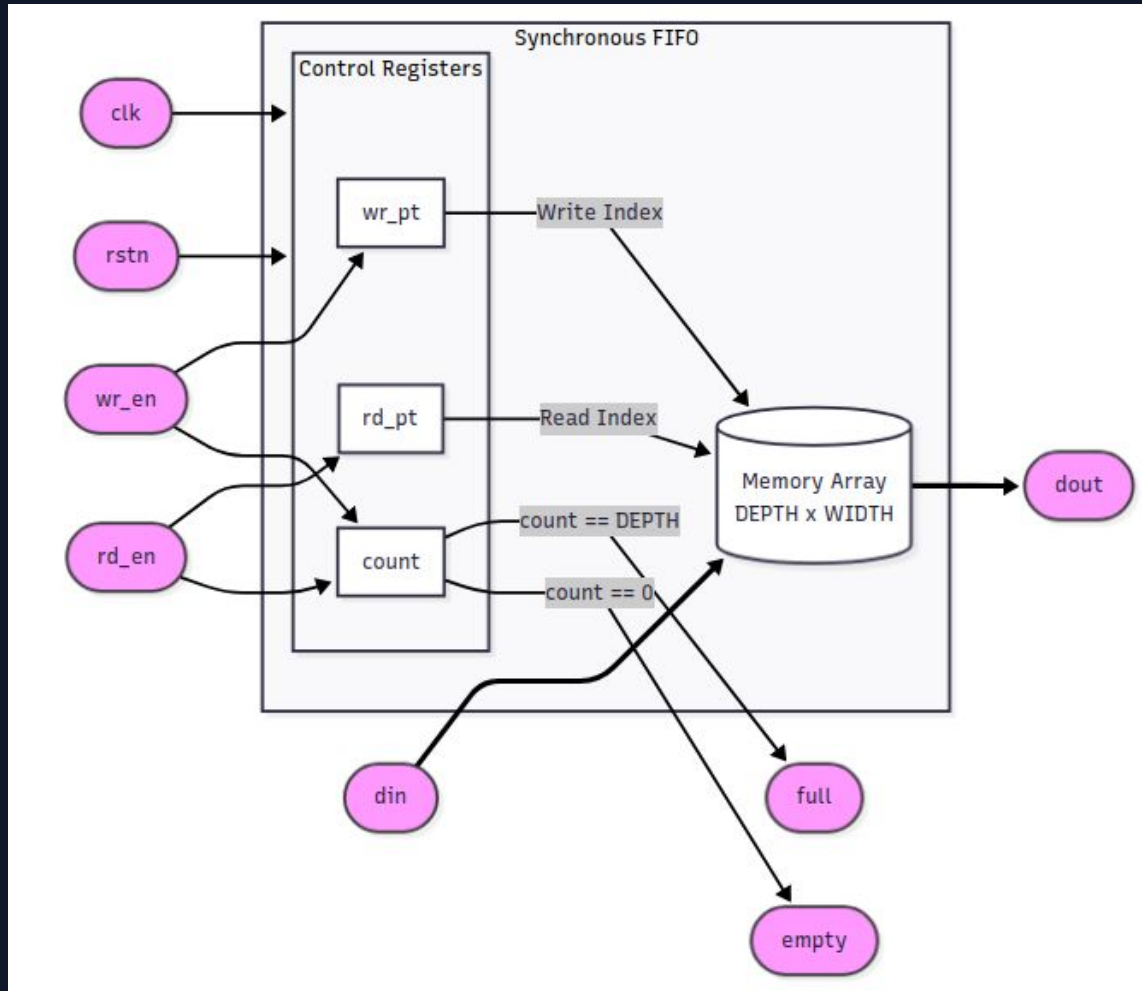
SYSTEM ARCHITECTURE

The environment uses a performance-optimized **hybrid approach**:

- ✓ **Simulation Engine:** Verilator (C++) handles the core clocking and high-speed DUT driving.
- ✓ **UVM Environment:** SystemVerilog testbench generates constrained-random sequences.
- ✓ **Coverage Channel:** DPI-C acts exclusively as a bridge to pass functional coverage data from SV.



DUT: SYNCHRONOUS FIFO



Design Specifications

A high-performance buffer utilizing a dual-port RAM structure:

- ✓ **Synchronous Operation:** Single clock domain for R/W.
- ✓ **Status Flags:** Full, Empty.
- ✓ **Error Handling:** Overflow and Underflow protection logic.
- ✓ **Parameters:** Configurable Depth (8) and Width (8).

UVM ENVIRONMENT



Agent & Sequencer

Handles the generation of constrained-random sequence items for FIFO transactions.



Coverage Collector

Listens for DPI-C function calls to record functional coverage points seamlessly.



Scoreboard

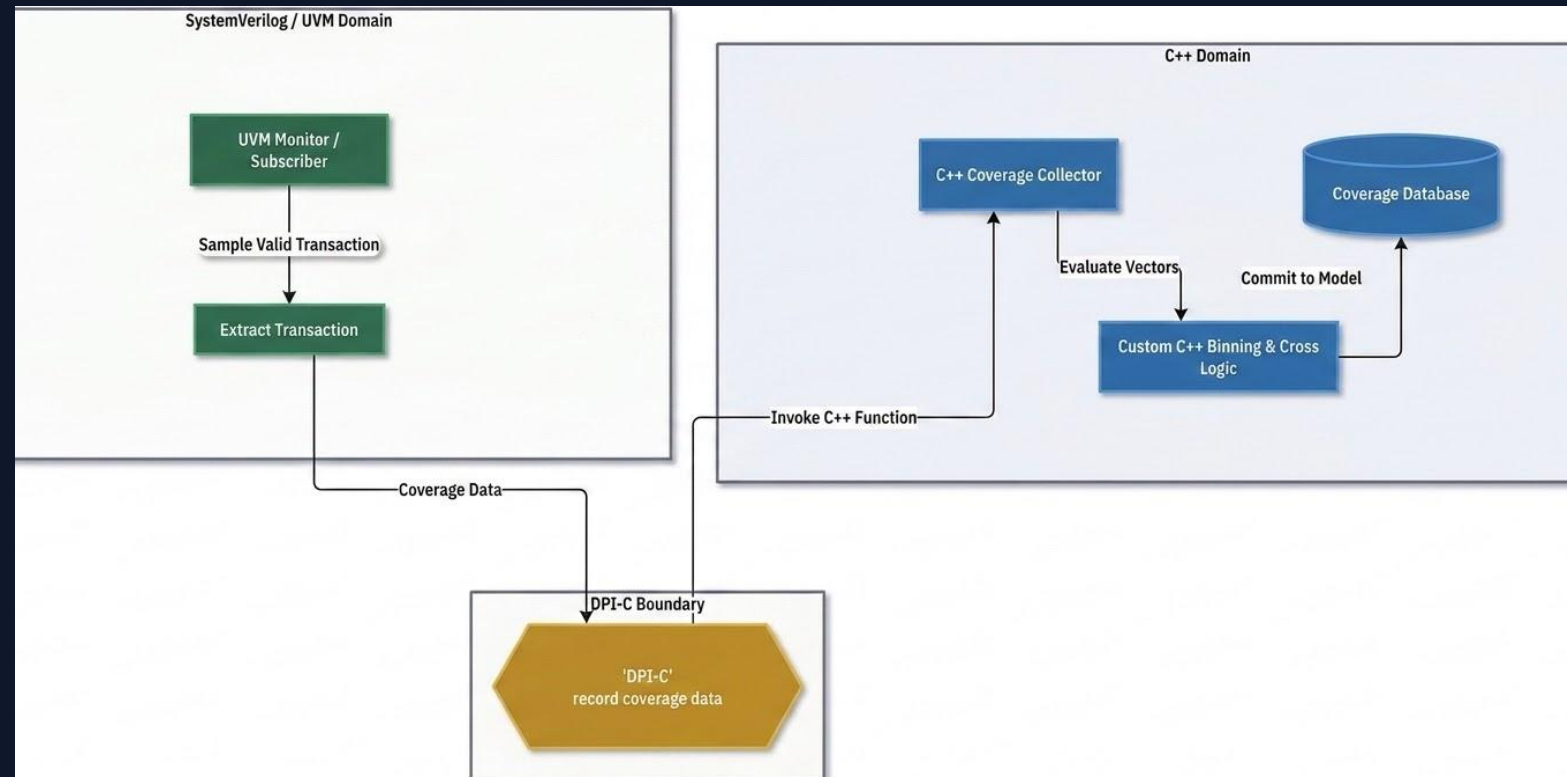
Self-checking mechanism comparing DUT output with an internal reference model.

THE DPI-C BRIDGE

Dedicated Coverage Channel

The DPI-C layer is exclusively utilized to pass functional coverage data from the SystemVerilog UVM environment to a C++ based coverage collector module.

By crossing the language boundary to offload the heavy lifting of SystemVerilog functional coverage, the architecture is a perfect workaround for Verilator's limitation with SV covergroup and bins constructs



VERIFICATION STRATEGY

- ✓ **Constrained Random Testing:** Generating millions of valid R/W combinations to find bugs.
- ✓ **Direct Test Cases:** Specifically targeting the "Full" and "Empty" flag transitions.
- ✓ **Corner Cases:** Simultaneous Read/Write operations at the boundary conditions.
- ✓ **Stability Testing:** Sustained throughput tests to ensure no data corruption over time.

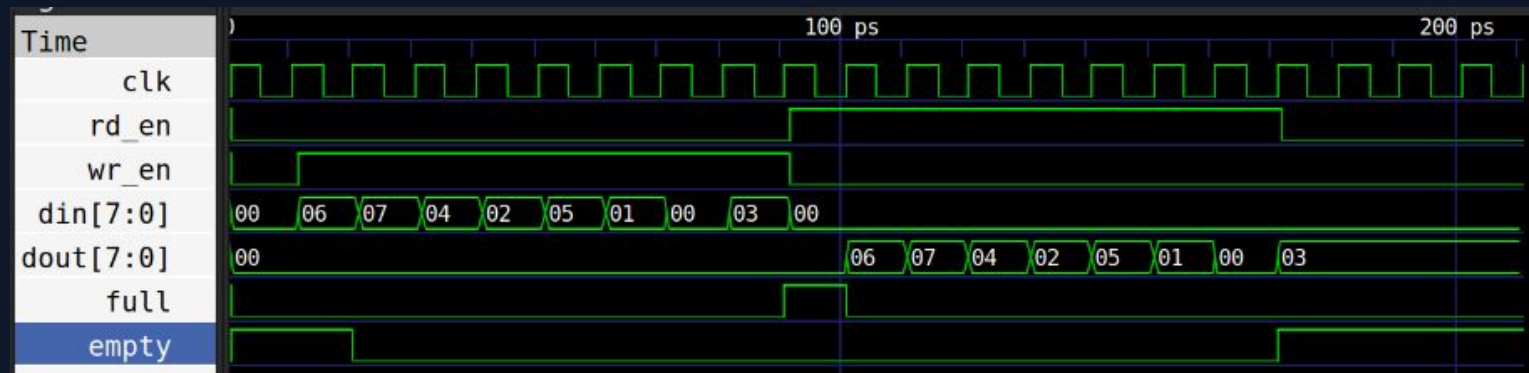
TEST MATRIX

Test Case	Objective	Status
FIFO_RESET_TEST	Verify initial state of flags and data pointers	PASSED
FIFO_OVERFLOW_TEST	Write to a full / read from an empty FIFO and check overflow / underflow flag	PASSED
FIFO_SIMUL_RW_TEST	Verify data integrity during concurrent R/W	PASSED
RANDOM_DATA_TEST	1000+ iterations of constrained-random data	PASSED

RESULTS & ANALYSIS

Simulation Highlights

- ✓ **Functional Coverage:** Achieved 100% on all cross-coverage points via DPI-C tracking.
- ✓ **Performance:** Verilator native execution provided vastly faster simulation compared to event-based tools.
- ✓ **Debugging:** Integrated with GTKWave for precise VCD signal tracing.



Questions?

Thank you for your attention.

Project Repository: github.com/NguyenThanhTai0307

Modern UVM Verification Flow with Open Source Tools